ROBOTIC IMITATION LEARNING

How a Task-Driven Scene Graph Works

A visual, step-by-step guide to Section 3.1 of Qian et al. (2026), with the released scene-graph-robot code as the implementation reference.

The central idea

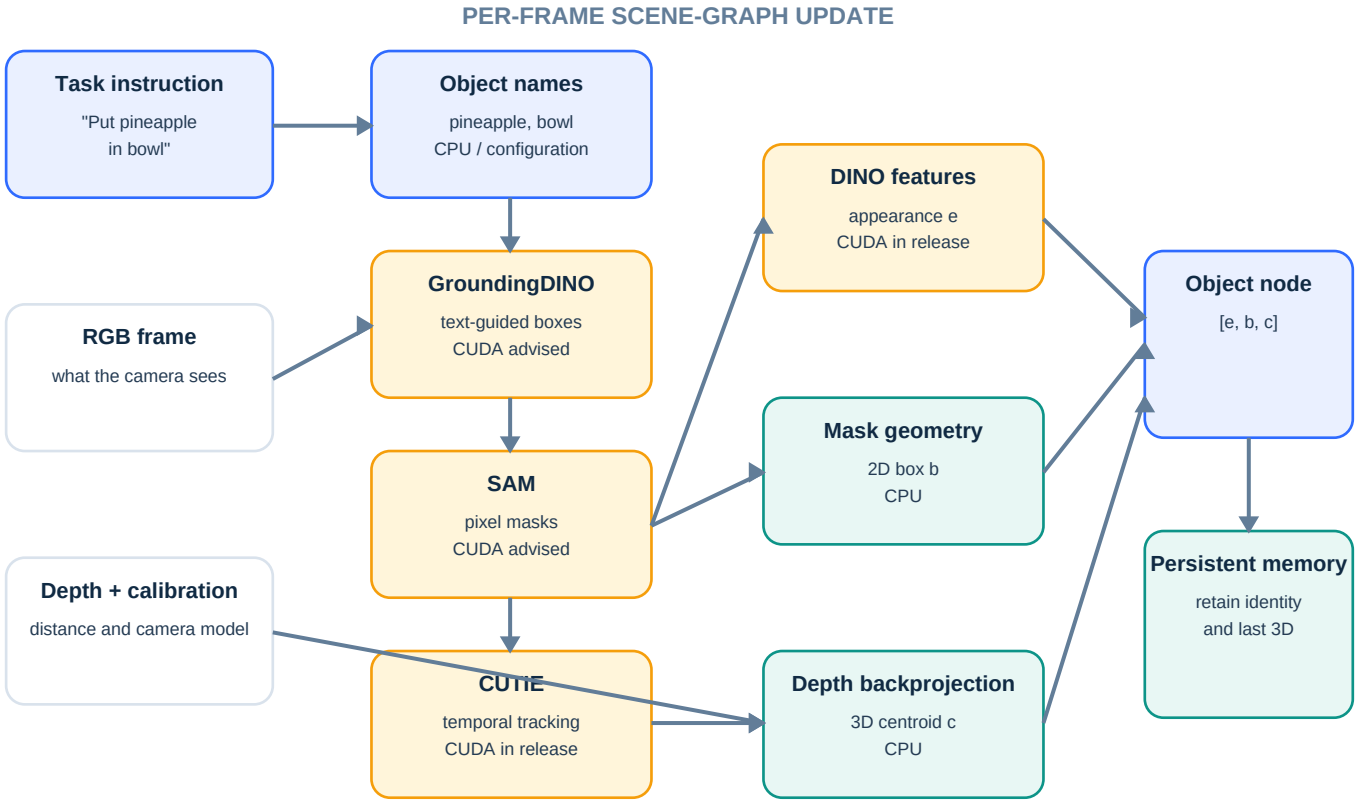
Turn a changing RGB-D camera stream into a small memory of task-relevant objects. Each object keeps an appearance vector, a current 2D image location, and a remembered 3D position.

You will learn	Practical outcome
How detection, segmentation, tracking, visual features, depth geometry, and memory connect	A clear mental model of the entire Section 3.1 pipeline
Which operations use CUDA	A staged plan that starts on a Mac and moves to an NVIDIA GPU only when needed
What the pineapple episode produces	The exact arrays that can later feed the transformer policy

Prepared for Vincent Pun | Johns Hopkins University | September 2026

1. The whole pipeline

Every frame follows the same chain: decide what matters, locate it, isolate its pixels, track its identity, measure its appearance and geometry, then update persistent memory.



One sentence to remember

GroundingDINO proposes **where** an object is, SAM identifies **which pixels** belong to it, CUTIE preserves **which object** it is over time, DINO describes **what it looks like**, and depth estimates **where it is in 3D**.

The output is one scene-level token plus a compact set of object nodes. Section 3.2 later passes those tokens into a transformer and diffusion policy; policy training is outside this guide.

2. From a task sentence to object masks

The pipeline begins by limiting perception to objects that can affect the task. For the instruction *Put the pineapple in the bowl*, the relevant entity list is simply **pineapple** and **bowl**.

Step 1 - Choose task-relevant names

- Paper method: ask a language model to extract important noun entities from the instruction.
- Released tasks: the names are already stored in each YAML configuration, so no language-model call is needed.
- Output: a short list of text prompts, such as [pineapple, bowl].

Step 2 - GroundingDINO proposes boxes

GroundingDINO receives the RGB image and one or more text prompts. It returns candidate rectangles and confidence scores for regions that match each name. A rectangle is only a rough localization; it often includes background pixels.

Step 3 - SAM converts each box into a mask

SAM uses the GroundingDINO rectangle as a spatial prompt and predicts a binary pixel mask. The mask separates the object from the table and nearby objects. The implementation then rejects implausible masks using confidence, area, overlap, and non-maximum-suppression rules.

Stage	Input	Output	Why it is needed
Object enumeration	Task instruction	Entity names	Prevents the graph from filling with irrelevant objects
GroundingDINO	RGB + entity name	Candidate box	Finds the named object in an open-vocabulary way
SAM	RGB + candidate box	Binary mask	Provides exact pixels for tracking, features, and geometry

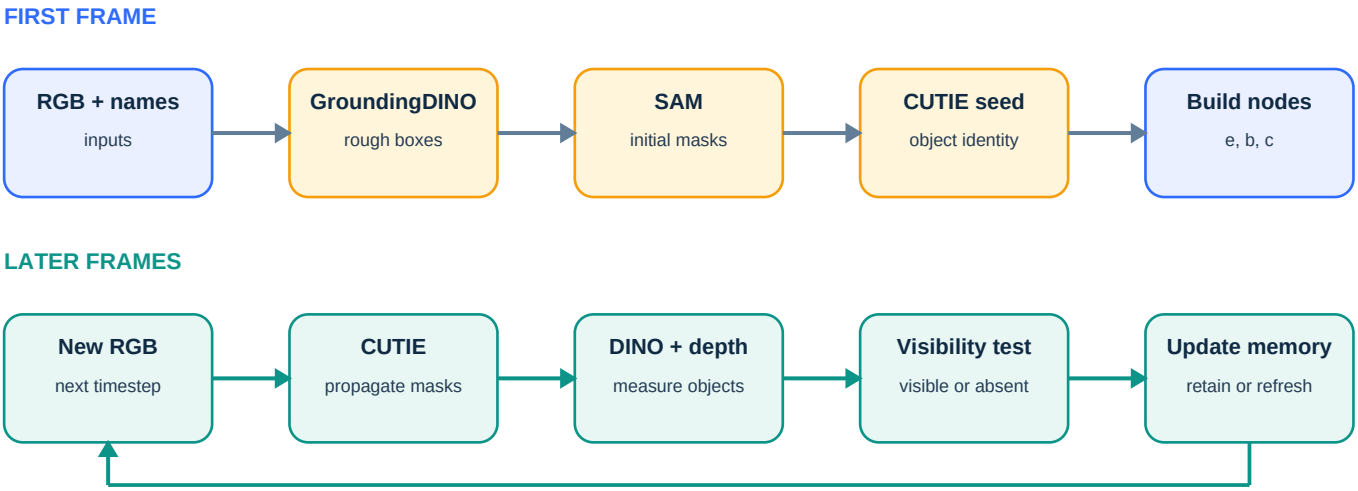
Failure to watch

A wrong entity list means the system never searches for the needed object. A poor detector box can seed the wrong SAM mask, and that error then propagates into tracking, visual features, and 3D position.

Suggested first check: annotate five RGB frames with GroundingDINO boxes and SAM masks. Confirm that pineapple pixels never leak heavily into the bowl mask, and vice versa.

3. Preserve identity through time

Object detection answers what is visible now. Scene memory also needs to know that the pineapple at time t=30 is the same pineapple seen at time t=0. CUTIE supplies that temporal identity.



How the temporal loop works

- At the first successful detection, the SAM mask initializes a CUTIE tracker for that semantic class.
- At later frames, CUTIE propagates the mask from prior visual evidence.
- If the tracked mask is non-empty, the pipeline measures a new appearance, 2D box, and 3D centroid.
- If the object is absent, the memory keeps its identity and last known 3D location.

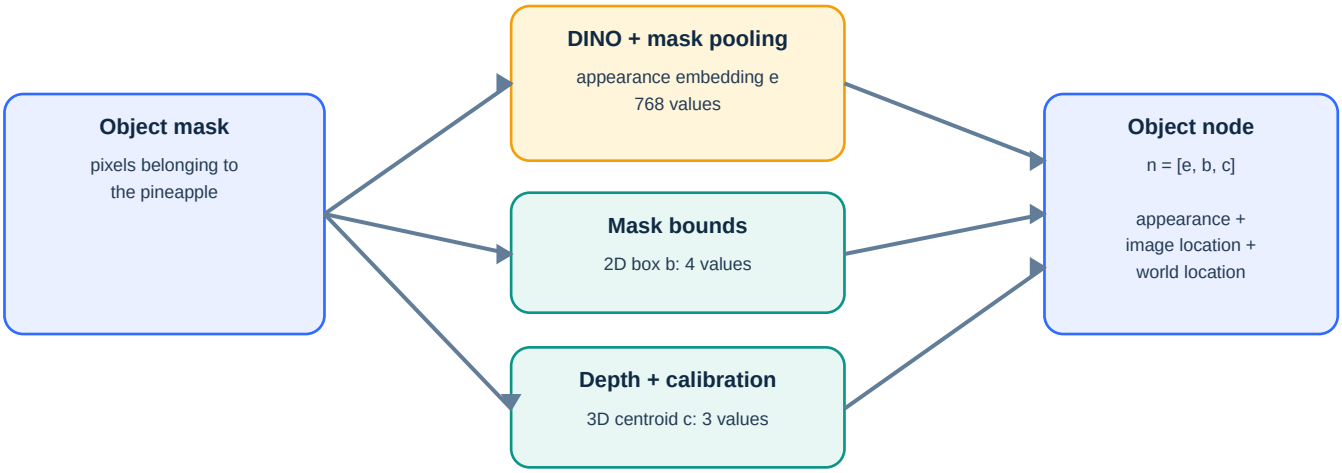
Repository behavior for pineapple

The supplied pineapple configuration skips late detection. If bowl or pineapple is missed during initialization, the released path may never create that track. Start with an episode where both are clear in the first processed frame, then improve re-detection after reproducing the baseline.

This tracking stage is where the representation becomes temporal. Without it, every frame would contain unrelated detections, so the system could not reliably decide what information to retain when an object disappears.

4. Build one object node

A mask is converted into three complementary measurements. Together they form the node n = [e, b, c].



Appearance embedding e

DINO converts the RGB image into a grid of visual feature vectors. The object mask is resized to that grid, and features inside the mask are average-pooled. The released encoder produces a 768-value vector. This lets the later transformer reason about appearance without processing every image pixel.

Visible 2D box b

The smallest rectangle enclosing the mask becomes [y_min, y_max, x_min, x_max], normalized by image height and width. It tells the policy where the object is in the current camera view. The box becomes zeros when the object is not visible.

World-aligned 3D centroid c

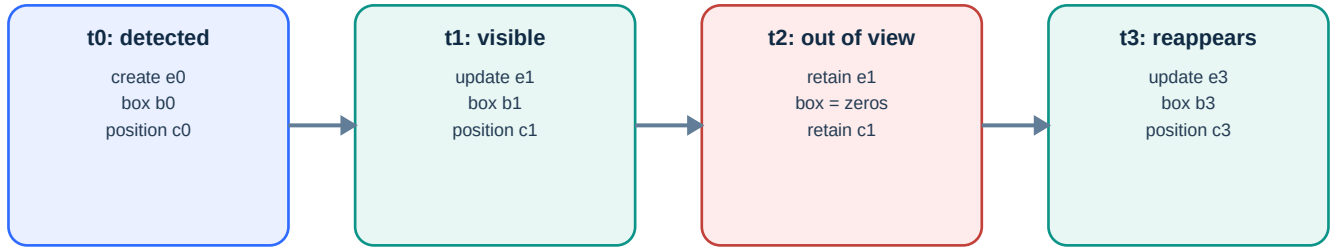
For each masked pixel (u, v) with depth Z, camera intrinsics recover X = (u - cx)Z/fx and Y = (v - cy)Z/fy. Extrinsics transform these camera points into the robot or world frame. A robust center of the valid points becomes c = [X, Y, Z].

Attribute	Meaning	Size	When invisible
e	Visual appearance	768	Retained in memory
b	Current image location	4	Set to [0, 0, 0, 0]
c	Last observed 3D location	3	Retained in memory

5. Turn measurements into memory

The graph is a shallow structure: one scene root and one node for each task-relevant object. Its most important behavior is the update rule used when visibility changes.

PINEAPPLE NODE OVER TIME



Memory bridges the period when the object is not visible.

Object state	Appearance e	2D box b	3D position c	Memory action
First detected	Create	Create	Create	Allocate an object node
Visible and tracked	Update	Update	Update	Refresh the node
Outside view	Retain	Set to zeros	Retain	Remember the last observation
Reappears	Update	Update	Update	Reconnect observation to identity

Pineapple graph at a single timestep

Slot	Node	Contents
0	Global scene	Full-frame visual feature + fixed scene geometry
1	Bowl	Bowl appearance + visible box + remembered 3D center
2	Pineapple	Pineapple appearance + visible box + remembered 3D center

Why this supports the research hypotheses

The visible-objects baseline discards a node when its object leaves view. Persistent memory retains e and c. The no-3D ablation removes c, testing whether remembering only appearance is enough for location-dependent actions.

6. Understand the saved representation

After processing an episode with T retained timesteps and C task classes, the repository saves two main time-indexed arrays. The extra slot is the global scene node.

Visual representation

episode_pr.npy
Shape: T x (C + 1) x 768

For pineapple, C = 2, so the shape is **T x 3 x 768**.

Geometry representation

episode_bboxes.npy
Shape: T x (C + 1) x 7

The seven values are [y_min, y_max, x_min, x_max, X, Y, Z]. For pineapple, the shape is **T x 3 x 7**.

Recommended validity metadata

The released arrays use zeros for several different states. Add explicit Boolean arrays so downstream experiments can distinguish them:

Mask	True means	Why it matters
seen_mask	The object has been detected at least once	Separates unknown objects from remembered objects
visible_mask	The object is visible in this frame	Separates current perception from memory
feature_valid_mask	The appearance embedding is trustworthy	Prevents missing features from looking like valid zeros
position_valid_mask	The 3D centroid used valid depth and calibration	Makes the no-3D ablation scientifically interpretable

Completion test for Section 3.1

You are finished with scene-graph construction when you can replay one episode and show: stable object identities, correct masks, synchronized RGB and depth, plausible 3D locations, and retained positions while an object is outside the camera view.

7. Where CUDA enters

The graph data structure and geometry do not depend on CUDA. The released neural perception path does: CUTIE and DINO call CUDA directly, while GroundingDINO and SAM are computationally heavy enough that GPU execution is the practical choice.

Operation	Role	CUDA status	Can you develop it on a Mac?
Choose object names	Defines graph vocabulary	No	Yes
Read RGB, depth, calibration	Loads observations	No	Yes
Bounding box from mask	Current 2D location	No	Yes
Depth backprojection	World-aligned 3D position	No	Yes
Persistent memory update	Retains identity and last position	No	Yes
GroundingDINO	Text-guided detection	Practically yes	Possible on CPU, but slow
SAM ViT-H	Pixel segmentation	Practically yes	Possible on CPU, but very slow
CUTIE	Temporal tracking	Required by released code	Requires refactor for CPU/MPS
DINO ViT-B/16	Visual embeddings	Required by released code	Requires device refactor

Recommended hardware for the GPU stage

- Linux workstation or cloud instance with an NVIDIA GPU.
- 24 GB VRAM recommended; 16 GB may work after reducing sizes or loading models sequentially.
- 32 GB system RAM and at least 50-100 GB free SSD space.
- A compatible NVIDIA driver, CUDA-enabled PyTorch build, and Git LFS.

Mac-first strategy

Use the Mac for data inspection, synchronization checks, geometry, memory logic, array validation, and visualizations. Move to CUDA only for GroundingDINO, SAM, CUTIE, and DINO inference. Avoid spending time on an Apple-MPS port until the CUDA reference path works.

8. A safe build sequence

Build confidence one dependency at a time. Do not start policy training until the scene representation passes visual and geometric checks.

Stage	Work	Estimated hands-on time	Exit criterion
1	Inspect one pineapple episode: RGB, depth, calibration, actions, and movement flags	2-4 hours	Counts and source indices are understood
2	Implement or test memory updates using manual or dummy masks	3-5 hours	Invisible objects retain identity and position
3	Validate RGB-depth synchronization and camera transforms	4-8 hours	Reprojection and stationary-object tests pass
4	Install CUDA PyTorch, GroundingDINO, SAM, CUTIE, and checkpoints	3-6 hours	Each model loads independently
5	Run a five-frame perception smoke test	2-4 hours	Boxes, masks, tracks, and features look correct
6	Process one complete pineapple episode	2-6 hours	Arrays and annotated video are produced
7	Add validity masks and package the clean representation	3-6 hours	Every zero and retained value has explicit meaning

Two checks required before trusting 3D

The released sample path filters disabled-motion trajectory steps but indexes depth using the filtered frame number, which can misalign RGB and depth. Its crop and padding operations also require a matching transformed camera intrinsic matrix. Preserve each original source index and validate the processed-camera model before using 3D results in hypothesis H2.

Key sources

Qian, J. et al. (2026). *Expanding Spatial and Temporal Context for Robotic Imitation Learning With Scene Graphs*. arXiv:2606.01072v2. https://arxiv.org/abs/2606.01072

Official repository: https://github.com/jianingq/scene-graph-robot, commit 69593b0f886ba6e9f77003176f180a5007bfa0f1, audited September 14, 2026.

Terminology note: the paper describes DINOv2 and a visual tracker; the current repository uses DINO ViT-B/16 and CUTIE. This guide labels the released implementation explicitly where it differs from the paper.